

Reinforcement Learning

Dr Stephen James
Autumn Term 2026
Imperial College London

Reinforcement Learning

Lecture 1: Markov Decision Processes

Dr Stephen James
Autumn Term 2026

Imperial College London

1. Introduction
2. Markov Decision Processes
3. Policies and Returns
4. Value Functions
5. Bellman Equations
6. Optimal Policies

1. Introduction
2. Markov Decision Processes
3. Policies and Returns
4. Value Functions
5. Bellman Equations
6. Optimal Policies

How does reinforcement learning actually work?

Intro: Why RL is amazing and will change the world



Lecture 1: How does RL actually work?

Prerequisites

- Linear algebra, probability theory, calculus

Learning Outcomes

- Understand what makes RL different from other ML
- Master the mathematical framework: MDPs
- Learn the core concepts: policies, value functions, Bellman equations

What mathematical tools do we need for RL?

Probability Essentials

- **Conditional probability:** $P(A|B) = \frac{P(A \cap B)}{P(B)}$
- **Chain rule:** $P(A \cap B) = P(A|B)P(B)$

Expectation

- **Definition:** $\mathbb{E}[X] = \sum_x x \cdot P(X = x)$
- **Linearity:** $\mathbb{E}[aX + bY] = a\mathbb{E}[X] + b\mathbb{E}[Y]$
- **Conditional:** $\mathbb{E}[X|Y] = \sum_x x \cdot P(X = x|Y)$
- **Law of total expectation:** $\mathbb{E}[X] = \mathbb{E}[\mathbb{E}[X|Y]]$

How does RL fit with other types of machine learning?

Paradigm	Learning Signal	Goal	Examples
Supervised	Labeled examples (x, y) pairs	Predict labels $y = f(x)$	Image classification Language translation
Unsupervised	Unlabeled data x only	Find patterns Density, structure	Clustering Dimensionality reduction
Reinforcement	Rewards (s, a, R, s')	Maximize return Optimal actions	Game playing Robot control

What makes RL uniquely challenging?

Key RL Challenges

- **Delayed consequences:** Actions have long-term effects
- **Exploration vs exploitation:** Try new things vs use what works
- **Credit assignment:** Which actions led to rewards?
- **Non-stationary:** Your actions change the data distribution

Why These Don't Exist in Supervised Learning

- **Immediate feedback:** Each (x, y) pair gives direct feedback
- **Fixed dataset:** No exploration needed, all data available
- **Clear attribution:** Each input directly maps to output
- **Stationary:** Data distribution doesn't change during training

1. Introduction
2. Markov Decision Processes
3. Policies and Returns
4. Value Functions
5. Bellman Equations
6. Optimal Policies

How do we formalize sequential decision making problems?

Definition (Markov Decision Process (MDP))

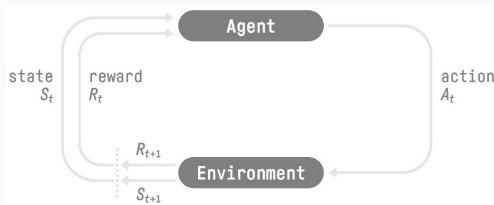
A model for sequential decision making when outcomes are uncertain.

Formally defined by the tuple $(\mathcal{S}, \mathcal{A}, P, R)$:

- $\mathcal{S}$: Set of **states** - all possible situations
- $\mathcal{A}$: Set of **actions** - all possible decisions
- $P(s'|s, a)$: **Transition probabilities** - how actions change states
- $R(s, a, s')$: **Reward function** - immediate feedback for transitions

If you can specify $(\mathcal{S}, \mathcal{A}, P, R)$, you can apply RL!

What does the agent-environment interaction look like?



The RL Loop:

1. Agent observes **state** s_t
2. Agent takes **action** a_t
3. Environment gives **reward** r_{t+1}
4. Environment transitions to new **state** s_{t+1}
5. Repeat...

Goal: Learn to maximize **cumulative reward**

Do tasks have natural start and end points?

Definition (Episode)

A complete sequence of interaction from start to terminal state:

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, \dots, S_T$$

where S_T is a terminal state.

Types of MDPs

- **Episodic:** Has clear start/end states (games, tasks)
 - Chess game, robot reaching goal, Atari game
- **Continuing:** No terminal states (ongoing processes)
 - Stock trading, server management, autonomous driving

Why this matters: Affects how we define returns and value functions

What do states, actions, and rewards look like in practice?

Atari Breakout

- States: ?
- Actions: ?
- Rewards: ?

Robot Manipulation

- States: ?
- Actions: ?
- Rewards: ?

Autonomous Driving

- States: ?
- Actions: ?
- Rewards: ?

Recommendation Systems

- States: ?
- Actions: ?
- Rewards: ?

What do states, actions, and rewards look like in practice?

Atari Breakout

- **States:** Pixel observations
- **Actions:** Left, Right, Fire
- **Rewards:** +1 for brick hit

Robot Manipulation

- **States:** ?
- **Actions:** ?
- **Rewards:** ?

Autonomous Driving

- **States:** ?
- **Actions:** ?
- **Rewards:** ?

Recommendation Systems

- **States:** ?
- **Actions:** ?
- **Rewards:** ?

What do states, actions, and rewards look like in practice?

Atari Breakout

- **States:** Pixel observations
- **Actions:** Left, Right, Fire
- **Rewards:** +1 for brick hit

Robot Manipulation

- **States:** ?
- **Actions:** ?
- **Rewards:** ?

Autonomous Driving

- **States:** Camera, lidar, GPS
- **Actions:** Steering, throttle, brake
- **Rewards:** Safe, efficient driving

Recommendation Systems

- **States:** ?
- **Actions:** ?
- **Rewards:** ?

What do states, actions, and rewards look like in practice?

Atari Breakout

- **States:** Pixel observations
- **Actions:** Left, Right, Fire
- **Rewards:** +1 for brick hit

Autonomous Driving

- **States:** Camera, lidar, GPS
- **Actions:** Steering, throttle, brake
- **Rewards:** Safe, efficient driving

Robot Manipulation

- **States:** Joint angles, object poses
- **Actions:** Joint torques/velocities
- **Rewards:** Task completion

Recommendation Systems

- **States:** ?
- **Actions:** ?
- **Rewards:** ?

What do states, actions, and rewards look like in practice?

Atari Breakout

- **States:** Pixel observations
- **Actions:** Left, Right, Fire
- **Rewards:** +1 for brick hit

Autonomous Driving

- **States:** Camera, lidar, GPS
- **Actions:** Steering, throttle, brake
- **Rewards:** Safe, efficient driving

Robot Manipulation

- **States:** Joint angles, object poses
- **Actions:** Joint torques/velocities
- **Rewards:** Task completion

Recommendation Systems

- **States:** User history, context
- **Actions:** Recommend items
- **Rewards:** Clicks, purchases, ratings

Why start with finite state and action spaces?

Definition (Finite MDP)

An MDP where both state and action sets are finite:

- $|\mathcal{S}| = n < \infty$ (finite number of states)
- $|\mathcal{A}| = m < \infty$ (finite number of actions)

Why Start With Finite MDPs?

- **Mathematical tractability:** Can represent everything as matrices/tables
- **Exact solutions:** Can find optimal policies exactly
- **Clear intuition:** Easy to visualize and understand
- **Foundation for continuous:** Continuous methods often discretize or generalize these ideas

How do we model uncertainty in state transitions?

Definition (Dynamics/Transition Function)

$$P(s'|s, a) = \Pr[S_{t+1} = s' | S_t = s, A_t = a]$$

The probability of transitioning to state s' given current state s and action a .

Key Properties

- **Probability distribution:** $\sum_{s' \in \mathcal{S}} P(s'|s, a) = 1$ for all s, a
- **Stationary:** Transition probabilities don't change over time
- **Markovian:** Next state depends only on current state and action

Deterministic Example:

- GridWorld: $P(s'|s, \text{Right}) = 1$
if s' is right of s

Stochastic Example:

- Sticky GridWorld: $P(s'|s, \text{Right}) = 0.9$
if s' is right, $P(s|s, \text{Right}) = 0.1$

Why is the Markov property so important?

Definition (Markov Property)

The future is independent of the past given the present:

$$P(S_{t+1}|S_t, a_t, S_{t-1}, a_{t-1}, \dots, s_0, a_0) = P(S_{t+1}|S_t, a_t)$$

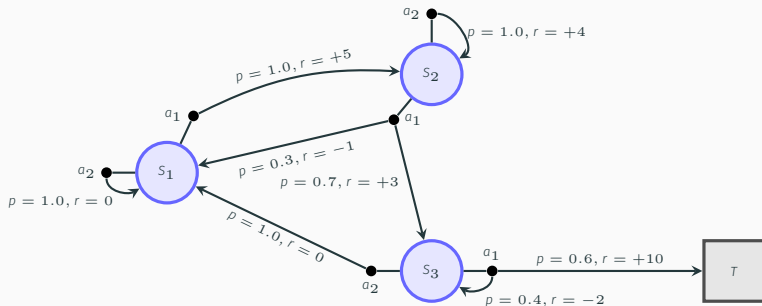
What This Means

- The current state contains all information needed to predict the future
- We don't need to remember the entire history
- Makes the problem mathematically tractable

Markovian vs Non-Markovian Examples

- **Markovian:** Tic-tack-toe, Chess (En Passant and Castling Rights included)
- **Non-Markovian:** Single Atari frame (no velocity), stock prices, poker (hidden cards)

How can we visualize MDP structure?



Transition Graph:

- Circles = states, Dots = actions (a_1 or a_2) the agent can choose
- Arrows out of a dot = possible next states, labelled with probability (p) and reward (r)
- The p 's leaving each dot sum to 1
- Square = terminal state (episode ends; no outgoing transitions)

How do we represent MDPs mathematically?

One transition matrix and one reward matrix **per action**:

$$P_{ij}^a = P(S_{t+1} = j \mid S_t = i, A_t = a) \quad R_{ij}^a = E[R_{t+1} \mid S_t = i, A_t = a, S_{t+1} = j]$$

		Action a_1			
		S_1	S_2	S_3	T
$\mathbf{P}^{a_1} =$	S_1	0.0	1.0	0.0	0.0
	S_2	0.3	0.0	0.7	0.0
	S_3	0.0	0.0	0.4	0.6

		S_1	S_2	S_3	T
$\mathbf{R}^{a_1} =$	S_1	·	+5	·	·
	S_2	-1	·	+3	·
	S_3	·	·	-2	+10

		Action a_2			
		S_1	S_2	S_3	T
$\mathbf{P}^{a_2} =$	S_1	1.0	0.0	0.0	0.0
	S_2	0.0	1.0	0.0	0.0
	S_3	1.0	0.0	0.0	0.0

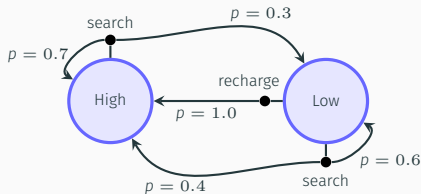
		S_1	S_2	S_3	T
$\mathbf{R}^{a_2} =$	S_1	0	·	·	·
	S_2	·	+4	·	·
	S_3	0	·	·	·

Each row of each $\mathbf{P}^a$ sums to 1.0 · · = transition cannot happen
No row for T : the episode ends there, so there are no transitions out of it



menti.com — Code: 1574 5831

A recycling robot's battery is **High** or **Low**. Which is $\mathbf{P}^{\text{search}}$?



A)

	High	Low
High	0.7	0.4
Low	0.3	0.6

B)

	High	Low
High	0.7	0.3
Low	1.0	0.0

C)

	High	Low
High	0.7	0.3
Low	0.4	0.6

D)

	High	Low
High	0.3	0.7
Low	0.6	0.4

menti.com — Code: 1574 5831

Clarity on MDPs?

- A) All topics clear
- B) RL Loop Unclear
- C) Transition Functions Unclear
- D) Transition Diagrams Unclear
- E) Markov Property Unclear

1. Introduction
2. Markov Decision Processes
3. Policies and Returns
4. Value Functions
5. Bellman Equations
6. Optimal Policies

How do agents choose actions?

Definition (Policy)

A policy π is a mapping from states to actions:

- **Deterministic:** $a = \pi(s)$
- **Stochastic:** $a \sim \pi(a|s)$

Example Policies:

- **Random:** Choose actions uniformly
- **Greedy:** Always go toward goal
- **Safe:** Avoid traps at all costs
- **Optimal:** Maximize cumulative reward

The goal of RL is to find the optimal policy π^* !

How do we measure total reward over time?

The Problem

How do we measure “total reward” when actions have long-term consequences?

Simple Example: Saving Money

- Today: Save \$100 (reward = $-\$100$)
- Tomorrow: Earn \$5 interest (reward = $+\$5$)
- Next day: Earn \$5 more interest (reward = $+\$5$)
- ...

Question: Was saving worth it? We need to sum ALL future rewards.

Why do we discount future rewards?

Definition (Discounted Return)

The return G_t is the cumulative discounted reward from time t :

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

Why Discounting? ($\gamma < 1$)

- **Mathematical convenience:** Ensures finite returns
- **Uncertainty:** Future rewards are less certain
- **Preference:** We prefer immediate rewards

Examples: $\gamma = 0$ (myopic), $\gamma = 0.9$ (balanced), $\gamma = 1$ (far-sighted)



menti.com - Code: 1574 5831

If $\gamma = 0$, what happens?

- A) The agent becomes more patient
- B) All states have the same value
- C) The Bellman equation becomes invalid
- D) Only immediate rewards matter
- E) The policy becomes deterministic

menti.com — Code: 1574 5831

Clarity on Policies and Returns?

- A) All topics clear
- B) Policies Unclear
- C) Returns Unclear

1. Introduction
2. Markov Decision Processes
3. Policies and Returns
4. Value Functions
5. Bellman Equations
6. Optimal Policies

How do we predict future success?

Definition (State Value Function)

The value of state s under policy π is the expected return:

$$V^\pi(s) = \mathbb{E}[G_t | s_t = s, \pi]$$

Definition (Action Value Function (Q-function))

The value of taking action a in state s under policy π :

$$Q^\pi(s, a) = \mathbb{E}[G_t | s_t = s, a_t = a, \pi]$$

Intuition

- $V^\pi(s)$: “How good is it to be in state s ?”
- $Q^\pi(s, a)$: “How good is it to take action a in state s ?”
- Both measure expected future reward, not immediate reward!

What do value functions look like visually?

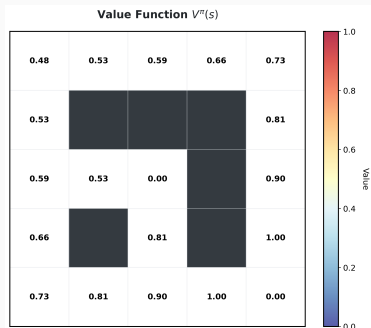


Simple 5×5 GridWorld

MDP Components:

- $\mathcal{S}$: Grid positions (i, j)
- $\mathcal{A}$: {Up, Down, Left, Right}
- P : Deterministic movement
- R : Goal=+1, Trap=-1, Step=0
- $\gamma = 0.9$

How do values spread through the state space?



State values $V^\pi(s)$ for a single policy (e.g. optimal policy)

What We See:

- **Goal state:** High value (close to +1)
- **Gradient:** Values decrease with distance from goal

Key Insight: Value functions tell us which states are 'good' to be in!

menti.com — Code: 1574 5831

Clarity on Value Functions?

- A) All topics clear
- B) Value Functions Unclear
- C) Action Value Functions Unclear
- D) Differences between V and Q Unclear

1. Introduction
2. Markov Decision Processes
3. Policies and Returns
4. Value Functions
5. Bellman Equations
6. Optimal Policies

What is the fundamental relationship in value functions?

Value functions satisfy a **recursive relationship** - we can define the value of a state in terms of the values of its successor states.

Theorem (Bellman Equation)

$$V^\pi(s) = \mathbb{E}[G_t | S_t = s, \pi] \quad \text{(expected return from state)}$$

$$= \mathbb{E}[R_{t+1} + \gamma G_{t+1} | S_t = s, \pi] \quad \text{(break into immediate + future)}$$

$$= \mathbb{E}[R_{t+1} + \gamma \mathbb{E}[G_{t+1} | S_{t+1}, \pi] | S_t = s, \pi] \quad \text{(linearity \& total expectation*)}$$

$$= \mathbb{E}[R_{t+1} + \gamma V^\pi(S_{t+1}) | S_t = s, \pi] \quad \text{(recursive value definition)}$$

$$= \sum_a \pi(a|s) \sum_{s', r} P(s', r|s, a) [r + \gamma V^\pi(s')] \quad \text{(expand expectation)}$$

$$\begin{aligned} * &= \mathbb{E}[R_{t+1} | S_t = s, \pi] + \gamma \mathbb{E}[G_{t+1} | S_t = s, \pi] && \text{(linearity)} \\ &= \mathbb{E}[R_{t+1} | S_t = s, \pi] + \gamma \mathbb{E}[\mathbb{E}[G_{t+1} | S_{t+1}, \pi] | S_t = s, \pi] && \text{(total expectation + Markov)} \\ &= \mathbb{E}[R_{t+1} + \gamma \mathbb{E}[G_{t+1} | S_{t+1}, \pi] | S_t = s, \pi] && \text{(linearity)} \end{aligned}$$

What is the fundamental relationship in value functions?

Value functions satisfy a **recursive relationship** - we can define the value of a state in terms of the values of its successor states.

Theorem (Bellman Equation)

$$\begin{aligned} V^\pi(s) &= \mathbb{E}[G_t | S_t = s, \pi] && \text{(expected return from state)} \\ &= \mathbb{E}[R_{t+1} + \gamma G_{t+1} | S_t = s, \pi] && \text{(break into immediate + future)} \\ &= \mathbb{E}[R_{t+1} + \gamma \mathbb{E}[G_{t+1} | S_{t+1}, \pi] | S_t = s, \pi] && \text{(linearity \& total expectation*)} \\ &= \mathbb{E}[R_{t+1} + \gamma V^\pi(S_{t+1}) | S_t = s, \pi] && \text{(recursive value definition)} \\ &= \sum_a \pi(a|s) \sum_{s', r} P(s', r|s, a) [r + \gamma V^\pi(s')] && \text{(expand expectation)} \end{aligned}$$

$$\begin{aligned} * &= \mathbb{E}[R_{t+1} | S_t = s, \pi] + \gamma \mathbb{E}[G_{t+1} | S_t = s, \pi] && \text{(linearity)} \\ &= \mathbb{E}[R_{t+1} | S_t = s, \pi] + \gamma \mathbb{E}[\mathbb{E}[G_{t+1} | S_{t+1}, \pi] | S_t = s, \pi] && \text{(total expectation + Markov)} \\ &= \mathbb{E}[R_{t+1} + \gamma \mathbb{E}[G_{t+1} | S_{t+1}, \pi] | S_t = s, \pi] && \text{(linearity)} \end{aligned}$$

What is the fundamental relationship in value functions?

Value functions satisfy a **recursive relationship** - we can define the value of a state in terms of the values of its successor states.

Theorem (Bellman Equation)

$$\begin{aligned} V^\pi(s) &= \mathbb{E}[G_t | S_t = s, \pi] && \text{(expected return from state)} \\ &= \mathbb{E}[R_{t+1} + \gamma G_{t+1} | S_t = s, \pi] && \text{(break into immediate + future)} \\ &= \mathbb{E}[R_{t+1} + \gamma \mathbb{E}[G_{t+1} | S_{t+1}, \pi] | S_t = s, \pi] && \text{(linearity \& total expectation*)} \\ &= \mathbb{E}[R_{t+1} + \gamma V^\pi(S_{t+1}) | S_t = s, \pi] && \text{(recursive value definition)} \\ &= \sum_a \pi(a|s) \sum_{s', r} P(s', r|s, a) [r + \gamma V^\pi(s')] && \text{(expand expectation)} \end{aligned}$$

$$\begin{aligned} * &= \mathbb{E}[R_{t+1} | S_t = s, \pi] + \gamma \mathbb{E}[G_{t+1} | S_t = s, \pi] && \text{(linearity)} \\ &= \mathbb{E}[R_{t+1} | S_t = s, \pi] + \gamma \mathbb{E}[\mathbb{E}[G_{t+1} | S_{t+1}, \pi] | S_t = s, \pi] && \text{(total expectation + Markov)} \\ &= \mathbb{E}[R_{t+1} + \gamma \mathbb{E}[G_{t+1} | S_{t+1}, \pi] | S_t = s, \pi] && \text{(linearity)} \end{aligned}$$

What is the fundamental relationship in value functions?

Value functions satisfy a **recursive relationship** - we can define the value of a state in terms of the values of its successor states.

Theorem (Bellman Equation)

$$\begin{aligned} V^\pi(s) &= \mathbb{E}[G_t | S_t = s, \pi] && \text{(expected return from state)} \\ &= \mathbb{E}[R_{t+1} + \gamma G_{t+1} | S_t = s, \pi] && \text{(break into immediate + future)} \\ &= \mathbb{E}[R_{t+1} + \gamma \mathbb{E}[G_{t+1} | S_{t+1}, \pi] | S_t = s, \pi] && \text{(linearity \& total expectation*)} \\ &= \mathbb{E}[R_{t+1} + \gamma V^\pi(S_{t+1}) | S_t = s, \pi] && \text{(recursive value definition)} \\ &= \sum_a \pi(a|s) \sum_{s', r} P(s', r|s, a) [r + \gamma V^\pi(s')] && \text{(expand expectation)} \end{aligned}$$

$$\begin{aligned} * &= \mathbb{E}[R_{t+1} | S_t = s, \pi] + \gamma \mathbb{E}[G_{t+1} | S_t = s, \pi] && \text{(linearity)} \\ &= \mathbb{E}[R_{t+1} | S_t = s, \pi] + \gamma \mathbb{E}[\mathbb{E}[G_{t+1} | S_{t+1}, \pi] | S_t = s, \pi] && \text{(total expectation + Markov)} \\ &= \mathbb{E}[R_{t+1} + \gamma \mathbb{E}[G_{t+1} | S_{t+1}, \pi] | S_t = s, \pi] && \text{(linearity)} \end{aligned}$$

What is the fundamental relationship in value functions?

Value functions satisfy a **recursive relationship** - we can define the value of a state in terms of the values of its successor states.

Theorem (Bellman Equation)

$$\begin{aligned} V^\pi(s) &= \mathbb{E}[G_t | S_t = s, \pi] && \text{(expected return from state)} \\ &= \mathbb{E}[R_{t+1} + \gamma G_{t+1} | S_t = s, \pi] && \text{(break into immediate + future)} \\ &= \mathbb{E}[R_{t+1} + \gamma \mathbb{E}[G_{t+1} | S_{t+1}, \pi] | S_t = s, \pi] && \text{(linearity \& total expectation*)} \\ &= \mathbb{E}[R_{t+1} + \gamma V^\pi(S_{t+1}) | S_t = s, \pi] && \text{(recursive value definition)} \\ &= \sum_a \pi(a|s) \sum_{s', r} P(s', r|s, a) [r + \gamma V^\pi(s')] && \text{(expand expectation)} \end{aligned}$$

$$\begin{aligned} * &= \mathbb{E}[R_{t+1} | S_t = s, \pi] + \gamma \mathbb{E}[G_{t+1} | S_t = s, \pi] && \text{(linearity)} \\ &= \mathbb{E}[R_{t+1} | S_t = s, \pi] + \gamma \mathbb{E}[\mathbb{E}[G_{t+1} | S_{t+1}, \pi] | S_t = s, \pi] && \text{(total expectation + Markov)} \\ &= \mathbb{E}[R_{t+1} + \gamma \mathbb{E}[G_{t+1} | S_{t+1}, \pi] | S_t = s, \pi] && \text{(linearity)} \end{aligned}$$

What is the fundamental relationship in value functions?

Value functions satisfy a **recursive relationship** - we can define the value of a state in terms of the values of its successor states.

Theorem (Bellman Equation)

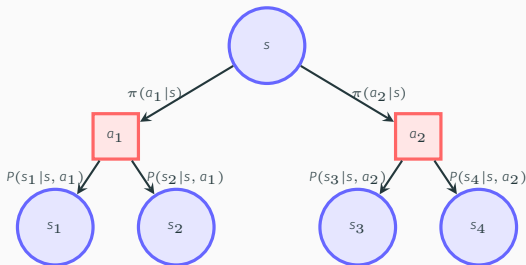
$$\begin{aligned} V^\pi(s) &= \mathbb{E}[G_t | S_t = s, \pi] && \text{(expected return from state)} \\ &= \mathbb{E}[R_{t+1} + \gamma G_{t+1} | S_t = s, \pi] && \text{(break into immediate + future)} \\ &= \mathbb{E}[R_{t+1} + \gamma \mathbb{E}[G_{t+1} | S_{t+1}, \pi] | S_t = s, \pi] && \text{(linearity \& total expectation*)} \\ &= \mathbb{E}[R_{t+1} + \gamma V^\pi(S_{t+1}) | S_t = s, \pi] && \text{(recursive value definition)} \\ &= \sum_a \pi(a|s) \sum_{s', r} P(s', r|s, a) [r + \gamma V^\pi(s')] && \text{(expand expectation)} \end{aligned}$$

$$\begin{aligned} * &= \mathbb{E}[R_{t+1} | S_t = s, \pi] + \gamma \mathbb{E}[G_{t+1} | S_t = s, \pi] && \text{(linearity)} \\ &= \mathbb{E}[R_{t+1} | S_t = s, \pi] + \gamma \mathbb{E}[\mathbb{E}[G_{t+1} | S_{t+1}, \pi] | S_t = s, \pi] && \text{(total expectation + Markov)} \\ &= \mathbb{E}[R_{t+1} + \gamma \mathbb{E}[G_{t+1} | S_{t+1}, \pi] | S_t = s, \pi] && \text{(linearity)} \end{aligned}$$

This equation is the foundation of ALL reinforcement learning algorithms!

How can we visualize the Bellman equation?

Backup Diagram for $V^\pi(s)$:



Key: States (circles), Actions (squares), Values flow backwards through tree

menti.com - Code: 1574 5831

The Bellman equation expresses:

- A) How to compute immediate rewards
- B) How to choose optimal actions
- C) The relationship between current and future values
- D) The transition probabilities
- E) The discount factor



menti.com - Code: 1574 5831

Which of these is **NOT** a correct step in deriving the Bellman equation?

A) $V^\pi(s) = \mathbb{E}[G_t | s_t = s, \pi]$

B) $V^\pi(s) = \mathbb{E}[R_{t+1} + \gamma G_{t+1} | s_t = s, \pi]$

C) $V^\pi(s) = \sum_a \pi(a|s) \sum_{s', r} P(s', r|s, a) \gamma[r + V^\pi(s')]$

D) $V^\pi(s) = \sum_a \pi(a|s) \sum_{s', r} P(s', r|s, a) [r + \gamma V^\pi(s')]$

1. Introduction
2. Markov Decision Processes
3. Policies and Returns
4. Value Functions
5. Bellman Equations
6. Optimal Policies

What defines the best possible policy?

Key Question

Among all possible policies, which one gives us the **highest expected return**?

Intuition: Some policies are better than others

- Policy A: Random actions → Low expected return
- Policy B: Go toward goal → Higher expected return
- Policy C: Go toward goal, avoid traps → Even higher!

What defines the best possible policy?

Key Question

Among all possible policies, which one gives us the **highest expected return**?

Definition (Comparing Policies)

Policy π is **better than or equal to** policy π' if:

$$V^{\pi}(s) \geq V^{\pi'}(s) \text{ for ALL states } s \in \mathcal{S}$$

We write this as: $\pi \geq \pi'$

Key insight: We compare policies by comparing their value functions!

What defines the best possible policy?

Key Question

Among all possible policies, which one gives us the **highest expected return**?

Definition (Optimal Policy π^*)

An optimal policy π^* satisfies:

$$\pi^* \geq \pi \text{ for all policies } \pi$$

In words: An optimal policy is **at least as good as** every other policy **in every state**.

Good news: For finite MDPs, at least one optimal policy always exists!

Even better: There may be multiple optimal policies, but they all achieve the same values.

Since all optimal policies achieve the same values, we can define:

Definition (Optimal State-Value Function)

$$V^*(s) \doteq \max_{\pi} V^{\pi}(s) \text{ for all } s \in \mathcal{S}$$

Definition (Optimal Action-Value Function)

$$Q^*(s, a) \doteq \max_{\pi} Q^{\pi}(s, a) \text{ for all } s \in \mathcal{S}, a \in \mathcal{A}$$

What these mean:

- $V^*(s)$: The **best possible** expected return starting from state s
- $Q^*(s, a)$: The **best possible** expected return starting from s , taking action a , then acting optimally

Relationship Between Optimal Functions

How do V^* and Q^* relate to each other?

$$V^*(s) = \max_a Q^*(s, a)$$

Intuition: The value of a state is the value of the **best action** from that state.

Going the other way: if we look one step ahead, we can write Q^* in terms of V^* :

$$Q^*(s, a) = \sum_{s', r} P(s', r | s, a) [r + \gamma V^*(s')]$$

Intuition: The value of an action is the **immediate reward** plus the **discounted value** of where you end up.

Relationship Between Optimal Functions

How do V^* and Q^* relate to each other?

$$V^*(s) = \max_a Q^*(s, a)$$

Intuition: The value of a state is the value of the **best action** from that state.

Going the other way: if we look one step ahead, we can write Q^* in terms of V^* :

$$Q^*(s, a) = \sum_{s', r} P(s', r | s, a) [r + \gamma V^*(s')]$$

Intuition: The value of an action is the **immediate reward** plus the **discounted value** of where you end up.

Question: If we know $Q^*(s, a)$, how do we act optimally?

Definition (Optimal Policy from Q^*)

$$\pi^*(s) = \operatorname{argmax}_a Q^*(s, a)$$

In words: In each state, pick the action with the highest Q^* value!

Why This Works

- $Q^*(s, a)$ tells us the long-term value of each action
- By definition, it already accounts for all future rewards
- So we can be “greedy”—just pick the best action right now
- This greedy policy is actually optimal for the long term!

How does this connect to modern deep reinforcement learning?

The Bridge from Classical to Modern

- Same math, different representation
- Tables $\rightarrow$ Neural networks
- Exact solutions $\rightarrow$ Approximate solutions
- Small problems $\rightarrow$ Complex problems

Tabular RL:

- $V(s)$ stored in table
- $Q(s, a)$ stored in table
- Exact Bellman updates
- Works for small $|\mathcal{S}|, |\mathcal{A}|$

Deep RL:

- $V(s) \approx V_{\theta}(s)$ neural net
- $Q(s, a) \approx Q_{\theta}(s, a)$ neural net
- Approximate Bellman updates
- Scales to huge state spaces

What are the key concepts we've learned today?

Key Takeaways

- **MDPs:** Formalize RL problems with states, actions, rewards
- **Policies:** Define agent's behavior, can be deterministic or stochastic
- **Returns:** Measure long-term success using discounted rewards
- **Value Functions:** Predict future success from states or state-action pairs
- **Bellman Equations:** Capture recursive relationships in value functions
- **Optimal Policies:** Achieve maximum expected return, defined by optimal value functions

Next Week: Dynamic Programming - How to compute optimal policies and value functions using Bellman equations



Essential Reading

- **Sutton & Barto:** Chapters 1 to 3 (MDPs and Bellman Equations)
- Focus especially on: Section 3.3 (Returns), 3.5 (Policies and Value Functions), 3.6 (Optimal Policies)



menti.com - Code: **1574 5831**

Q1: If you got lost today, at what point did you lose track?

- A) Markov property and MDPs
- B) Bellman equations for state values
- C) Bellman equations for action values
- D) Backup diagrams
- E) Optimal value functions and policies
- F) All concepts are clear

Q2: Generic feedback - what worked well, what could be improved?

- e.g. "I found the section on policies particularly helpful."
- e.g. "Pace was too fast."
- e.g. "Not enough examples."